

Horus: A modular GPU emulator framework

Amr Saleh Elhelw, ECE department, University of Rochester (aelhelw@ur.Rochester.edu)

Sreepathi Pai, CS department, University of Rochester (sree@cs.Rochester.edu)

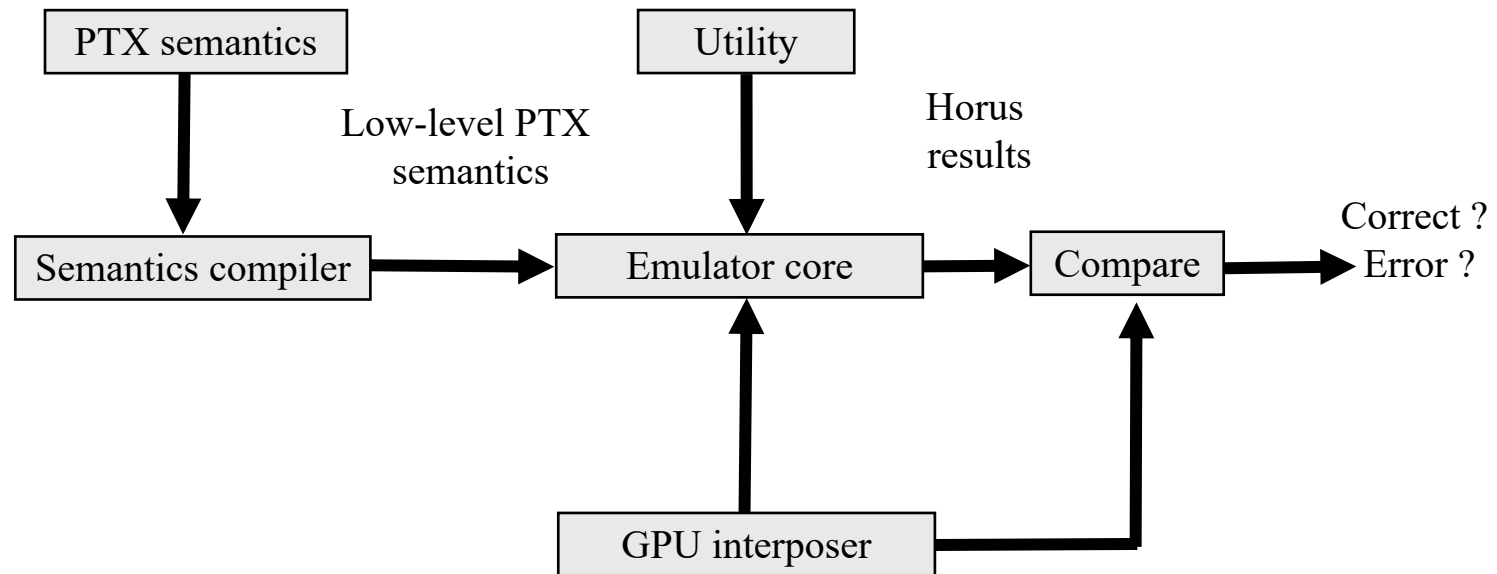


What Horus is

- Horus is:
 - GPU PTX Emulator written in Python
- Horus components are:
 1. GPU interposer
 2. PTX semantics
 3. Semantics compiler
 4. Emulator core
 5. Utility component



Horus block diagram



GPU interposer

- Intercepts application's interaction with the GPU
- Record traces using LTTNG
- Traces record all API calls, their arguments, and data moved between CPU and GPU.



Emulator core

- Uses a recorded trace to execute an application
- Generates loops based on number of blocks/warps/threads required by an application
- Integrates low-level executable semantics and utility functions to execute PTX instructions
- Compares the simulated results against hardware results and generates an error report



PTX semantics

- High level semantics for each PTX instruction
- PTX semantics are divided into scalar and vector semantics
- Scalar semantics
 - Thread-level semantics
 - One type instructions, such as **mul24**
 - Multiple types instructions such as **add/sub**
 - Memory: such as **load/store**
- Vector semantics
 - Multilevel semantics such as barriers and branch instructions



Add PTX Semantics

- Both *add.u32 a, b, c* and *add.f64 a, b, c* have the same operation ($a = b + c$)
- Only instruction type and bitwidth are different (integer/float)
- Same PTX semantics represents similar instructions
- High-level add semantics is shown below (like Python)
- High-level semantics is compiled to low-level executable semantics

```
def execute_add(sign, sat, cc, roundModifier, bitWidth, dst, src1, src2, number,
single_precision_src1, double_precision_src1, pred_inst_reg, invert, regFile, cc_reg,
varType, thread_pc_count):
    dst = src1 + src2
```



Semantics compiler

- Written in Python, and based on Python built in Abstract Syntax Tree (AST)
- Generates executable function for each PTX instruction based on its semantics and arguments
- Compiler adds all required type checks and Utility functions

```
def execute_add(sign, sat, cc, roundModifier, bitWidth, dst, src1, src2, ...):  
    ..... #check whether the instruction is predicated or not  
    thread_pc_count = (thread_pc_count + 1)  
    if ((number == None) and (double_precision_src1 == None)): number = single_precision_src1  
    elif ((number == None) and (single_precision_src1 == None)): number = double_precision_src1  
    else: number = number  
  
    tmp_dst = add(set_sign_bitWidth(regFile[src1], sign, varType, bitWidth),  
                  (set_sign_bitWidth(regFile[src2], sign, varType, bitWidth) if (src2 != None) else  
                  number), varType, bitWidth)  
    if cc: cc_reg.cf = int((regFile[dst] > ((2 ** bitWidth) - 1)))  
    tmp_dst_2 = set_round(tmp_dst, roundModifier)  
    regFile[dst] = set_sign_bitWidth(tmp_dst_2, sign, varType, bitWidth)  
    return thread_pc_count
```

Highlighted functions are utility functions



Utility

- Emulator behavior could be changed by changing any function in utility
- Such as **set_round**, **set_sign_bitWidth**, ... etc
- Add utility is shown below

```
def add(value1, value2, varType, bitWidth):  
    if (varType == 'Float'):  
        return float(value1 + value2)  
    elif (varType == 'Integer'):  
        return (value1 + value2)  
    else:  
        print (varType + ' is not implemented')  
        exit(0)
```



Evaluation

- Individual PTX instructions are compared against real GPU (Quadro P5000) and GPGPU-Sim
 - Horus results matched Nvidia Quadro P5000 results
 - GPGPU-Sim had bugs in **mad/fma**
- Horus executed successfully Polybench and Rodinia benchmarks excluding benchmarks which use **tex**
- Horus was on average 2x slower than GPGPU-Sim
- Speeding up Horus is our future work

